

Plan para huir de DiSí

Sep 30, 2026 · @Alex

Cómo usar este plan

En 12 semanas pasas de entregar con IA a poder explicar y escribir tú mismo lo que entregas, mientras la búsqueda de empleo corre en paralelo desde la semana 1. El plan asume unas **10 horas por semana de estudio**: 1.5 h de lunes a jueves y 4 h el sábado; con la búsqueda de empleo del viernes (1 h) y el carril de repos del domingo (45 min), la semana completa suma unas 12 horas (11 h 45 min). Cada semana tiene dos carriles: teoría (el sábado y el lunes, 5.5 h), que recorre el árbol de fundamentos de abajo hacia arriba, y código (martes a jueves, 4.5 h), que practica en C# lo que la teoría acaba de explicar. Los domingos hay un carril corto para aprender a leer y presentar repositorios de GitHub (45 min).

Reglas del plan

1. **Tú escribes, la IA enseña.** Pídele explicaciones, revisiones de tu código y preguntas de examen, nunca el código. Si no puedes explicar una línea, no está terminada.
2. **Nada de copiar y pegar.** Si necesitas un ejemplo, léelo, ciérralo y escríbelo de memoria.
3. **Todo va a GitHub.** Tres repos: plan-dotnet (ejercicios de las semanas 1 a 5, una carpeta por semana), SolicitudesApi (tu API de portafolio, semanas 6 a 11) y GcalPlan (semana 8). Commits pequeños y mensajes claros. Es tu evidencia.
4. **Explica en voz alta.** Cierra cada sesión explicando en 2 minutos lo que aprendiste, como si fuera una entrevista. Grábate de vez en cuando.
5. **Glosario propio.** Cada término nuevo va a la nota 08 Glosario de tu bóveda de Obsidian, con tu definición y un ejemplo de tu trabajo en DiSí.

Prompt para tu tutor de IA: "Actúa como mi mentor de .NET. No me des código. Explícame el concepto, hazme 3 preguntas y revisa mi solución señalando errores sin corregirlos por mí."

Recursos base (gratis): Microsoft Learn (rutas de C# y ASP.NET Core), la documentación oficial de .NET en learn.microsoft.com, y ejercicios de C# en Exercism.

Vista general: 12 semanas, 3 fases

Empieza el lunes 5 de octubre y termina el 27 de diciembre de 2026. La teoría sigue el [árbol de fundamentos](#) y el código lo aplica en la misma semana o la siguiente.

Sem.	Inicia	Teoría (sáb. + lun.)	Código en C# (mar.-jue.)	Entregable
1	5 oct	0.1 Datos y 0.2 Hardware	Instalar SDK, <code>dotnet CLI</code> , tipos, variables, control de flujo, métodos	5 ejercicios de consola en GitHub
2	12 oct	0.3 Sistema operativo y 0.4 Programas (CLR, stack y heap)	Clases, objetos, valor vs. referencia, <code>struct</code> , <code>record</code>	Modelo <code>Cliente</code> / <code>Credito</code> con pruebas manuales
3	19 oct	1. Redes e Internet	Interfaces, herencia, polimorfismo, colecciones, genéricos	<code>ICredito</code> con <code>CreditoSimple</code> y <code>CreditoRevolvente</code>
4	26 oct	2. HTTP	LINQ, excepciones, nullable	Reporte de cartera con LINQ
5	2 nov	3. Navegador y 4. APIs y REST	<code>async/await</code> , <code>HttpClient</code> , JSON	Consola que consume una API pública y maneja errores
6	9 nov	5. Criptografía y 6. HTTPS	ASP.NET Core: <code>Program.cs</code> , middleware, DI, configuración; SHA-256 y HMAC en C#	API de solicitudes de crédito, primeros endpoints
7	16 nov	7. Identidad y tokens	Controllers, DTOs, validación, EF Core, migraciones, SQL Server	API con persistencia real
8	23 nov	OAuth 2.0 (la clase)	Proyecto Google Calendar: OAuth de escritorio, crear calendario y eventos	Tu plan agendado en Google Calendar por tu propio código
9	30 nov	SQL Server a fondo	JWT en tu API, <code>[Authorize]</code> , <code>ProblemDetails</code> , logging	API protegida por roles

Sem.	Inicia	Teoría (sáb. + lun.)	Código en C# (mar.-jue.)	Entregable
10	7 dic	Resiliencia y webhooks	Webhook de Twilio con firma HMAC, <code>IHttpClientFactory</code> , Polly, idempotencia	Webhook funcionando y documentado
11	14 dic	Git, Docker y Azure (conceptos)	xUnit, <code>docker compose</code> , GitHub Actions, README	Repo de portafolio terminado
12	21 dic	Repaso de todo el árbol	Live coding cronometrado y simulacros	Guion de tus proyectos de DiSí

Si una semana se atrasa, no la saltes: mueve todo una semana. Construir sobre una base floja es justo lo que queremos evitar.

Fase 1 (semanas 1-3): la computadora por dentro y C# base

Meta: entender qué pasa dentro de la máquina cuando corre tu código, y volver a escribir C# a mano. Todo en consola con `dotnet new console`, sin plantillas de Visual Studio. El IDE es Visual Studio 2026 con .NET 10 (LTS); en las semanas 1 y 2 creas y compilas los proyectos con el CLI para entender qué hace el IDE por ti, y mantienes apagadas las sugerencias de Copilot durante todo el plan.

Cómo se conectan teoría y código:

- Semana 1: aprendes que un `int` son 4 bytes y que `double` aproxima → en el código compruebas `0.1 + 0.2` con `double` y con `decimal`, y por qué el dinero va en `decimal`.
- Semana 2: aprendes stack y heap → en el código asignas un `struct` y una `class` a otra variable, cambias un campo y explicas por qué uno cambia y el otro no.
- Semana 3: aprendes puertos y localhost → corres `netstat -ano` mientras levantas un proyecto de prueba y ves el puerto abierto.

Preguntas de cierre de fase:

- ¿Qué diferencia hay entre un proceso y un hilo?
- ¿Qué pasa en memoria al asignar un tipo por valor y uno por referencia?
- ¿Cómo pasa tu código C# de texto a instrucciones de CPU?
- ¿Qué ocurre, capa por capa, cuando escribes una URL y presionas Enter? (hasta TCP)

- ¿Cuándo usarías una interfaz y cuándo una clase abstracta?

Fase 2 (semanas 4–8): la web, la seguridad, ASP.NET Core y OAuth

Meta: entender HTTP y la seguridad web desde la base, levantar una API en .NET 10 desde cero y cerrar con la integración de Google Calendar escrita por ti.

Cómo se conectan teoría y código:

- Semana 4–5: aprendes HTTP y REST → consumes una API con `HttpClient` y lees cada header y código de estado que recibes.
- Semana 6: aprendes hash y HMAC → calculas SHA-256 y HMAC en C#; es exactamente lo que harán PKCE (semana 8) y el webhook de Twilio (semana 10).
- Semana 6–7: arrancas la **API de solicitudes de crédito** en ASP.NET Core. Prepara tu tabla de .NET Framework vs. .NET 10: `Global.asax` vs. `Program.cs`, `web.config` vs. `appsettings.json`, IIS vs. Kestrel, DI externo vs. integrado.
- Semana 8: con los 8 niveles del árbol (0 a 7) ya estudiados, relees la [clase de OAuth](#) y programas el proyecto de Google Calendar: autenticación, crear el calendario "Plan .NET" y agendar este mismo plan.

Preguntas de cierre de fase:

- ¿Qué diferencia hay entre 401 y 403? ¿Y entre PUT y PATCH?
- ¿Por qué Base64 no es cifrado y un JWT está firmado pero no cifrado?
- ¿Qué pasa si inyectas un servicio `Scoped` dentro de uno `Singleton`?
- ¿Por qué importa el orden del middleware?
- Explica el Authorization Code flow con PKCE usando tu proyecto de Google Calendar.

Fase 3 (semanas 9–12): producción, pruebas y entrevistas

Meta: dejar la API lista para mostrarla en entrevista y practicar hasta explicar todo con soltura.

Semana	Teoría	Código
9	SQL Server: joins, índices clustered y non-clustered, transacciones y aislamiento, planes de ejecución	JWT en tu API: login que emite tokens, endpoints por rol (analista, admin), <code>ProblemDetails</code> , <code>ILogger</code>

Semana	Teoría	Código
10	Resiliencia: timeouts, reintentos, idempotencia, rate limiting; webhooks	Webhook de Twilio que valida <code>X-Twilio-Signature</code> , guarda con EF Core y no procesa duplicados
11	Git (ramas, rebase vs. merge, pull requests), Docker, Azure App Service y Azure SQL	Pruebas con xUnit y <code>WebApplicationFactory</code> , <code>docker compose</code> con API + SQL Server, GitHub Actions
12	Repaso del árbol completo en voz alta	2 simulacros técnicos, 1 live coding de 45 minutos, guion STAR de tus 5 proyectos de DiSí

Live coding: desde la semana 5, dedica 30 minutos del sábado a un ejercicio fácil de Exercism o LeetCode en C#, sin IA. En México suelen pedir ejercicios de CRUD o de lógica de negocio más que algoritmos difíciles, pero hay que llegar con soltura.

Carril extra: leer y presentar repositorios de GitHub

Un repositorio público no se evalúa leyendo todo el código. Quien lo revisa sigue un recorrido de 5 a 10 minutos y busca señales de cómo trabajas. Este carril te enseña a hacer ese recorrido en repos ajenos y a preparar el tuyo para que lo pase. Son **45 minutos cada domingo**: 30 de lectura y 15 de la revisión semanal. Con este carril la semana completa suma unas 12 horas.

Qué mira alguien que revisa tu repo

Orden	Dónde mira	Qué busca	Qué dice de ti
1	README	Qué es, qué problema resuelve, cómo correrlo, capturas o demo, decisiones técnicas	Que sabes comunicar y pensar en quien usa tu trabajo
2	Estructura de carpetas	<code>src/</code> y <code>tests/</code> separados, nombres claros, un <code>.sln</code> ordenado	Que organizas el código con criterio

Orden	Dónde mira	Qué busca	Qué dice de ti
3	Historial de commits	Commits pequeños y frecuentes con mensajes descriptivos, no "cambios" o "fix"	Cómo trabajas día a día y si el proyecto es tuyo de verdad
4	Pruebas	Que existan, que tengan nombres que expliquen el caso y que pasen	Que te importa la calidad
5	CI (GitHub Actions)	Un workflow que compila y corre pruebas; la palomita verde	Que conoces prácticas profesionales
6	Punto de entrada y un flujo	<code>Program.cs</code> y un endpoint seguido hasta la base de datos	Cómo separas responsabilidades
7	Seguridad básica	Sin secretos en el repo, <code>.gitignore</code> correcto, configuración por entorno	Que eres confiable con código de producción
8	Perfil	README de perfil, 4 a 6 repos fijados, descripciones y <i>topics</i>	Qué quieres que vean primero

Los reclutadores no técnicos solo leen el README y el perfil. Los técnicos abren además commits, pruebas y un flujo de código. En entrevista es común que te pidan compartir pantalla y recorrer tu propio repo.

Temario, domingo por domingo

Sem.	Domingo	Tema	Práctica
1	11 oct	Anatomía de un repo: pestañas Code, Issues, Pull requests, Actions, Releases	Recorrer el README y las carpetas de <code>dotnet/eShop</code>
2	18 oct	Historial: commits, diff, <i>blame</i> , mensajes de commit	Leer 10 commits de <code>ardalis/CleanArchitecture</code> ; adoptar Conventional Commits en tu repo

Sem.	Domingo	Tema	Práctica
3	25 oct	Soluciones .NET: <code>.sln</code> , <code>.csproj</code> , <code>src/</code> y <code>tests/</code> , <code>Directory.Build.props</code>	Mapear la estructura de <code>jasontaylordev/CleanArchitecture</code>
4	1 nov	Issues y pull requests: descripción, revisión, checks de CI	Leer un PR cerrado de <code>App-vNext/Polly</code> de principio a fin
5	8 nov	Seguir un flujo: de un endpoint a la base de datos	Abrir <code>dotnet/eShop</code> en <code>github.dev</code> (tecla punto) y seguir un pedido con "Ir a definición"
6	15 nov	Qué hace bueno a un README	Comparar 3 READMEs y reescribir el de tu API
7	22 nov	Tu perfil de GitHub	Crear tu README de perfil, fijar repos, agregar descripciones y <i>topics</i>
8	29 nov	Leer un SDK	En <code>googleapis/google-api-dotnet-client</code> , encontrar dónde se implementa el flujo OAuth que acabas de usar
9	6 dic	GitHub Actions por dentro	Leer los workflows en <code>.github/workflows</code> de un repo y entender cada paso
10	13 dic	Seguridad en código ajeno	En <code>twilio/twilio-csharp</code> , encontrar <code>RequestValidator</code> y comparar con tu validación de firma
11	20 dic	Revisión como reclutador	Pedirle a Claude que revise tu repo con la tabla de arriba y corregir lo que falte
12	27 dic	Presentar tu repo en entrevista	Guion de 3 minutos para recorrer tu repo en pantalla compartida y grabarte

Método para leer cualquier repo: README, estructura de carpetas, archivo de solución y proyectos, punto de entrada, un flujo completo, pruebas y, al final, commits recientes. Anota en 01 Bitacora tres cosas que harías igual en tu repo y una que harías distinto.

Búsqueda de empleo en paralelo

Como el plan dura 12 semanas, las entrevistas fuertes caen entre finales de noviembre y mediados de diciembre. La contratación suele bajar en la segunda quincena de diciembre y repunta en enero, así que ese es tu segundo momento fuerte.

Semanas	Acciones
1-3	CV con logros medibles, LinkedIn en español e inglés, lista de 20 empresas objetivo (fintech, banca, SOFOMes, consultoras .NET). Todavía no aplicas
4-7	3-5 aplicaciones por semana a vacantes de menor prioridad, como práctica. Anota cada pregunta que no supiste contestar y agrégala a tu bitácora
8-10	Aplicaciones a tus empresas objetivo, ya con el proyecto de Google Calendar y la API en el CV
11-12	Seguimiento de procesos abiertos y negociación. Pide 30k netos y confirma si es nómina con prestaciones u honorarios
Enero	Segunda ola de aplicaciones con el portafolio completo

Registro: lleva en la nota 09 Registro de aplicaciones de Obsidian una tabla con empresa, puesto, fecha, etapa, salario ofrecido (bruto o neto) y preguntas que te hicieron.

Checklist de salida (semana 12)

- ☐ Explico los 8 niveles del árbol, del bit a los tokens, con un ejemplo propio de cada uno
- ☐ Cuento completo "qué pasa cuando escribes una URL y presionas Enter"
- ☐ Levanto una Web API en .NET 10 desde cero sin IA en menos de 1 hora
- ☐ Explico DI, middleware, `async/await` y EF Core sin titubear
- ☐ Mi consola agenda eventos en Google Calendar con OAuth, escrita por mí
- ☐ Recibo un webhook con validación de firma y justifico cada código HTTP de mi API
- ☐ Repo público con README, pruebas, Docker y CI, y perfil de GitHub con repos fijados; lo recorro en pantalla compartida en 3 minutos

- ☐ Guion de 5 proyectos de DiSí: problema, flujo, decisiones, errores y resultado
- ☐ 08 Glosario con al menos 100 términos propios
- ☐ 2 simulacros técnicos aprobados y al menos 15 aplicaciones enviadas